

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM

Desenho Hierárquico de Grafos com Minimização de Cruzamentos Utilizando o Método Parallel Tempering

Bolsista: Vinícius Nunes dos Anjos

Orientador: Marco Antônio Moreira de Carvalho - DECOM/UFOP

Nota: Relatório Final referente ao período de 01/03/2026 a 31/08/2026, apresentado à Universidade Federal de Ouro Preto, como parte das exigências do programa de iniciação científica do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) Brasil.

Local: Ouro Preto - Minas Gerais - Brasil

Data: 17 de setembro de 2026

Resumo

Este relatório apresenta a modelagem, implementação e avaliação experimental da metaheurística *Parallel Tempering* (PT) aplicada ao problema de Minimização de Cruzamentos Unilaterais (*One-Sided Crossing Minimization* – OSCM) em grafos bipartidos hierárquicos. O OSCM é um problema NP-difícil essencial no arcabouço de Sugiyama para visualização de dados, cujo objetivo é encontrar uma ordenação linear para os vértices de uma camada livre de modo a minimizar as interseções de arestas contra uma camada fixa, reduzindo a carga cognitiva na interpretação de diagramas.

A metodologia empregada fundamenta-se na adaptação de uma API em C++ de alto desempenho voltada para arquiteturas *multi-core*, desacoplando os operadores específicos do problema da orquestração paralela das réplicas e do controle térmico estocástico. A abordagem incorpora heurísticas construtivas baseadas no Baricentro e na Mediana, aliadas a operadores de perturbação (*Insert*, *Swap* e *2-opt*) e esquemas dinâmicos de ajuste de temperatura balizados pela taxa ótima teórica de aceitação de 23%. Para mitigar o expressivo custo computacional associado à contagem de cruzamentos, desenvolveu-se um mecanismo híbrido de avaliação: uma rotina analítica parcial de complexidade $O(1)$ suportada por matriz de cruzamentos sob alocação dinâmica e avaliação preguiçosa (*lazy evaluation*), combinada a uma avaliação global exata baseada em árvores de Fenwick com segurança de concorrência por *threads*.

A validação experimental foi realizada utilizando o *benchmark* oficial do desafio global PACE 2024, composto por 600 instâncias divididas igualmente entre as trilhas Exata, Parametrizada e Heurística. O desempenho do método PT (executado com 4 réplicas em regimes de 5 minutos e 1 hora) foi confrontado com o algoritmo genético memético (GA) do time CIMAT, vencedor da competição. No regime restrito de 5 minutos, o algoritmo proposto igualou ou superou o GA em 66,8% do total de instâncias avaliadas, registrando um *gap* médio global residual de apenas 0,751%. Com a ampliação do tempo de execução para 1 hora, a contínua troca térmica entre réplicas preveniu a estagnação prematura em ótimos locais, elevando o percentual de equivalência/superioridade para 72,2% e contraindo o *gap* médio para 0,036%. Destacou-se a trilha parametrizada, na qual o PT alcançou 98,5% de convergência absoluta à solução de referência do estado da arte. Os resultados comprovam a solidez teórica e a eficiência computacional do PT como uma ferramenta de alta escalabilidade para otimização topológica em grafos massivos.

Assinatura do orientador(a): _____
Marco Antonio Moreira de Carvalho

Assinatura do bolsista: _____
Vinícius Nunes dos Anjos

Sumário

1	Introdução	1
1.1	Justificativa	3
1.2	Organização do Trabalho	3
2	Objetivos	4
3	Revisão da Literatura	5
3.1	Análise Metodológica do PACE 2024	7
3.1.1	Trilha Exata	7
3.1.2	Trilha Parametrizada	7
3.1.3	Trilha Heurística: Estratégias de Busca e Metaheurísticas	7
4	Materiais e Métodos	9
4.1	Minimização de Cruzamentos Unilaterais	9
4.2	Revenimento paralelo	11
5	Desenvolvimento	14
5.1	A API de Revenimento Paralelo	14
5.2	Codificação e Decodificação	14
5.3	Heurística Construtiva da Solução Inicial	15
5.4	Operadores de Perturbação e Vizinhança	15
5.5	Funções de Avaliação	15
5.5.1	Funções de Avaliação Completa	16
5.5.2	Avaliação Parcial	16
5.6	Distribuição Inicial de Temperaturas	16
5.7	Ajuste Dinâmico de Temperaturas	17
5.8	Critério de Parada	17
6	Resultados e Discussão	19
6.1	Instâncias	19
6.2	Ajuste de Parâmetros e Protocolo Experimental	19
6.3	Avaliação das Instâncias Exatas	20
6.4	Avaliação das Instâncias Parametrizadas	21
6.5	Avaliação das Instâncias Heurísticas	22
6.6	Discussão Integrada e Análise de Escalabilidade	23
7	Conclusões	25

Capítulo 1

Introdução

Os grafos são abstrações matemáticas de grande relevância para estudo de relações entre entidades em um sistema. São utilizados para modelar e resolver uma grande variedade de problemas na Ciência da Computação, Engenharias, Ciências Naturais e Sociais. O primeiro registro do uso formal de grafos vem do século XVIII [Euler, 1741], em que se discutiu a solução do problema das “sete pontes de Königsberg”. Desde então, a Teoria dos Grafos evoluiu de uma curiosidade topológica para um pilar da computação moderna, enfrentando desafios contemporâneos.

Formalmente, um grafo $G = (V, E)$ é definido por um conjunto não vazio de vértices V e um conjunto de arestas E , que representam, respectivamente, as entidades do sistema e as relações estabelecidas entre elas. Cada aresta é responsável por conectar um par de vértices e, quando essa conexão ocorre, diz-se que tais vértices são adjacentes. A representação visual de um grafo é denominada diagrama, como ilustrado na Figura 1.1, que apresenta a modelagem do problema das sete pontes de Königsberg. Neste grafo, há 4 vértices rotulados de A a D e sete arestas conectando os vértices, permitindo uma análise clara do sistema.

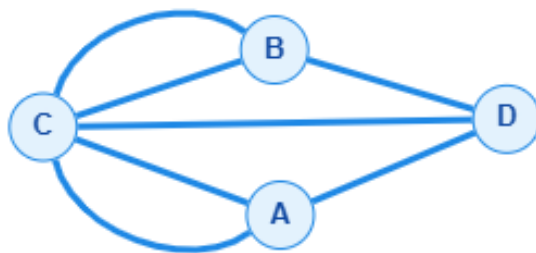


Figura 1.1: Diagrama do problema das Pontes de Königsberg, uma abstração de um sistema geográfico em um modelo em grafos.

Os desafios contemporâneos da área se diversificam em várias perspectivas. Com o aumento exponencial do volume e complexidade dos dados, questões relacionadas à visualização e à representação geométrica eficiente tornaram-se cruciais para a interpretabilidade e processamento dos grafos. Atualmente, a relevância dos grafos é potencializada pela intensa digitalização de processos e pelo advento do *Big Data* [Pires Magalhaes et al., 2015], consolidando-os como uma estrutura fundamental para representar desde redes sociais e fluxos logísticos até complexas interações biológicas.

Conforme discutido por Mishra et al. [2023], a análise de grafos massivos exige uma abordagem integrada de sumarização, interpretação e visualização. Nesse contexto, a escalabilidade e o aumento da densidade dos dados impõem barreiras críticas, pois tendem a gerar oclusão e ruídos que comprometem a eficácia da representação visual. Assim, o desenvolvimento de métodos que otimizem a legibilidade dessas estruturas torna-se vital para viabilizar análises eficientes e o suporte a sistemas automatizados.

Nesse contexto, apesar da robustez dos grafos como modelos puramente matemáticos, um diferencial reside, fundamentalmente, na capacidade de representá-los visualmente de forma clara. O campo de desenho de grafos surge dessa necessidade [Tutte, 1963], fazendo a junção da geometria espacial e teoria dos grafos para criar diagramas que facilitem a percepção humana para captação da informação desejada. Como mencionado, uma representação visual mal planejada, caracterizada pela sobreposição excessiva de elementos e cruzamentos de arestas desnecessários, pode ocultar informações cruciais, tornando a interpretação do modelo ineficiente ou até errônea.

O desenho hierárquico de grafos é a representação visual de grafos que possuem uma estrutura intrínseca de estratificação dos vértices. Segundo Healy and Nikolov [2013], essa representação organiza os vértices em uma sequência de camadas horizontais, como pode-se ver na Figura 1.2, de modo que as arestas estejam dispostas em uma direção comum, para outra camada adjacente. Nessa estrutura em particular forma-se um grafo bipartido, onde o conjunto de vértices é particionado em duas camadas e todas as adjacências ocorrem exclusivamente entre elas, jamais internamente. No referido grafo bipartido, uma camada é composta pelos vértices A1 a A8, e a outra camada é composta pelos vértices B1 a B8.

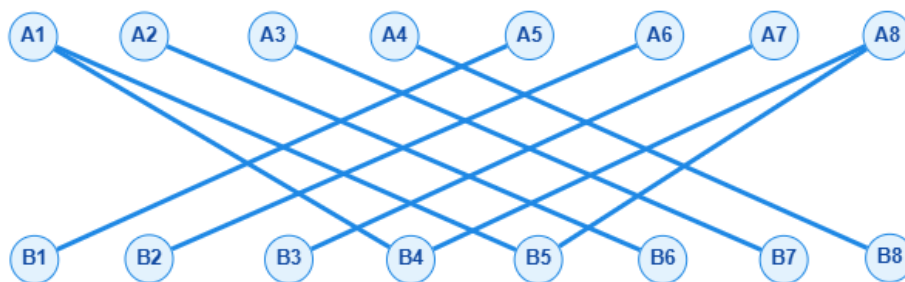


Figura 1.2: Exemplo de um grafo bipartido em camadas com alta densidade de cruzamentos

Dentre as métricas de eficácia do desenho de grafos, a minimização do número de cruzamentos entre arestas consolida-se como um dos principais critérios de qualidade estética e funcional. Estudos experimentais pioneiros [Tamassia, 1990, Warfield, 2007] na área de interação humano-computador corroboram que a interseção de arestas é o fator que mais prejudica a legibilidade de um diagrama, uma vez que eleva significativamente o esforço cognitivo necessário para o rastreamento de adjacências entre os vértices. Na Figura 1.2, pode observar-se esse fenômeno, pois a disposição atual dos vértices gera excesso de cruzamentos entre as arestas, com 3 a 5 cruzamentos em cada uma das arestas, totalizando 22, o que dificulta a identificação imediata das adjacências. Nesse sentido, a busca por representações visuais que minimizem essas interseções transcende o desafio puramente matemático, focando na otimização da percepção visual.

Este estudo aborda a minimização de cruzamentos de arestas entre camadas adjacentes em um grafo bipartido. Trata-se do problema conhecido como *One-Sided Crossing Minimization* (OSCM). Busca-se reduzir as interseções de arestas ao reordenar os vértices de uma camada

enquanto a outra permanece fixa. Mesmo neste caso, trata-se de um problema NP-Difícil [Eades and Wormald, 1994]. Essa característica implica que, para grafos de grande escala, encontrar a solução ótima por meio de algoritmos exatos torna-se inviável, demandando a aplicação de heurísticas e metaheurísticas para obtenção de boas soluções subótimas.

Para abordagem ao OSCM, emprega-se uma implementação do método *Parallel Tempering* (PT), também conhecido como revenimento paralelo. Originalmente concebido para a simulação de sistemas termodinâmicos complexos na física e química molecular [Hukushima and Nemoto, 1996, Hansmann, 1997], o PT fundamenta-se na execução de buscas paralelas em diferentes intensidades, permitindo que o algoritmo alterne entre a exploração global do espaço de soluções e o refinamento local. Assim, o uso do PT apresenta-se como uma ferramenta promissora para otimizar a legibilidade de grafos hierárquicos em cenários de alta complexidade, onde a busca pela solução ótima é computacionalmente proibitiva.

1.1 Justificativa

O OSCM é um problema amplamente pesquisado na atualidade, mantendo-se relevante devido à sua complexidade intrínseca e aplicabilidade em visualização de dados. Uma das principais motivações recentes para o avanço desses estudos foi a realização do *Parameterized Algorithms and Computational Experiments* [PACE, 2024] — um desafio global voltado à implementação de algoritmos exatos, heurísticos e parametrizados para a solução do problema em grafos bipartidos. O objetivo central proposto pelo desafio foi o desenvolvimento de soluções eficientes para instâncias de grande escala, incentivando a comunidade acadêmica a avançar o estado da arte sobre o tema.

A relevância estratégica do OSCM reside no fato de ele ser um subproblema fundamental na “Abordagem de Sugiyama” [Sugiyama et al., 1981], o *framework* mais difundido para o desenho de grafos multicamadas. Este método resolve o OSCM de forma iterativa para pares de camadas adjacentes, realizando varreduras sucessivas com o intuito de reduzir o número total de cruzamentos em toda a estrutura. Portanto, otimizações no tratamento do caso unilateral refletem-se diretamente na qualidade estética e funcional de diagramas hierárquicos complexos.

Por se tratar de um problema NP-Difícil [Eades and Wormald, 1994], a escolha do PT como método de solução justifica-se pela sua arquitetura intrinsecamente paralela, uma característica rara entre métodos de otimização combinatória baseados em trajetórias. Conforme indicam Earl and Deem [2005], embora o método seja uma ferramenta padrão nas áreas de física e química, sua natureza generalista permite a aplicação em diferentes domínios. Portanto, este trabalho contribui para o avanço científico ao transpor uma técnica robusta de simulação para a visualização de dados, utilizando implementações modernas que exploram o paralelismo real para obter ganhos expressivos de desempenho na solução do OSCM.

1.2 Organização do Trabalho

Este trabalho está organizado conforme a estrutura descrita a seguir. O Capítulo 2 apresenta os objetivos geral e específicos deste estudo. O Capítulo 3 é dedicado à revisão dos trabalhos relacionados. O Capítulo 4 trata da fundamentação teórica, abordando tanto o problema OSCM quanto o método utilizado. Em seguida, o Capítulo 5 detalha a metodologia empregada e o desenvolvimento da solução. O Capítulo 6 relata os experimentos computacionais e os resultados obtidos a partir das instâncias fornecidas pelo PACE. Por fim, as conclusões finais deste estudo são apresentadas no Capítulo 7.

Capítulo 2

Objetivos

O objetivo geral deste trabalho consiste em aplicar e avaliar o método PT na resolução do OSCM. Busca-se explorar o potencial desse método para reduzir o esforço cognitivo na visualização de grafos hierárquicos, visando obter soluções de alta qualidade para instâncias de grande escala. Para isso, foram estabelecidos as seguintes etapas:

1. Realizar uma revisão bibliográfica sobre o OSCM, com foco em sua relevância para o desenho de grafos e nas abordagens heurísticas existentes na literatura;
2. Consolidar o embasamento teórico acerca do método PT, compreendendo seus mecanismos de trocas paralelas e sua aplicabilidade em problemas de otimização;
3. Desenvolver e implementar uma versão eficiente do algoritmo PT, adaptada às restrições e à função objetivo do OSCM;
4. Avaliar o desempenho do método proposto por meio de experimentos computacionais, utilizando as instâncias de teste fornecidas pelo desafio *PACE 2024*;
5. Analisar comparativamente a eficácia da abordagem em termos de redução do número de cruzamentos e tempo de processamento frente aos outros métodos conhecidos.

Capítulo 3

Revisão da Literatura

No estudo de Tutte [1963], estabelece-se a conjectura de que a legibilidade do diagrama de um grafo está diretamente associada ao número de cruzamentos de suas arestas, definindo que o posicionamento dos vértices no plano rege a clareza da representação visual. Sob essa premissa geométrica, a formulação do OSCM evoluiu de uma abordagem estritamente algébrica para um modelo consolidado de otimização topológica.

Inicialmente, Harary [1969] estabeleceu uma conexão teórica basilar ao demonstrar que o número de cruzamentos em grafos bipartidos correspondia diretamente a propriedades de matrizes de adjacência binárias, vinculando a topologia do grafo ao cálculo de determinantes e permanentes. Esse conceito ganhou contornos puramente geométricos com Watkins [1970], que introduziu o modelo visual moderno de desenho de grafos ao propor a disposição dos vértices sobre linhas retas paralelas. Esse processo de formalização culminou nos trabalhos de Harary [1972], que cunharam formalmente o problema clássico *2-layer crossing minimization* sob o cenário de reordenação livre de ambas as camadas, pavimentando o caminho para o OSCM.

A transição desse modelo geral de duas camadas variáveis para a restrição de lado único tornou-se uma necessidade algorítmica com o advento dos grafos hierárquicos multicamadas. Nesse contexto, Sugiyama et al. [1981] introduziram um método seminal para a visualização de sistemas complexos, estabelecendo um *framework* estruturado em fases consecutivas: remoção de ciclos, atribuição de camadas, minimização de cruzamentos e definição de coordenadas geométricas. Para gerenciar a interdependência entre as múltiplas camadas na fase de cruzamentos, os autores propuseram uma estratégia iterativa de varredura. Sob esse paradigma, ao isolar duas camadas adjacentes, a ordem de uma delas é mantida fixa enquanto a outra é permutada, estabelecendo formalmente o OSCM. O referido trabalho introduziu também a heurística do baricentro, inaugurando a linhagem de soluções focadas em médias locais.

A fundamentação teórica da complexidade do desenho de grafos foi estabelecida por Garey and Johnson [1979, 1983], que provaram que o problema de decisão para a minimização de cruzamentos é NP-completo. Direcionando a análise para o caso unilateral (OSCM), Eades and Wormald [1994] formalizaram que sua otimização é NP-difícil em sua forma geral, uma intratabilidade que Muñoz et al. [2001] demonstraram persistir rigorosamente mesmo sob condições de esparsidade estrita. Dada essa barreira para a obtenção de soluções ótimas exatas, o desenvolvimento de aproximações tornou-se a via analítica primária. Nesse sentido, os próprios autores Eades and Wormald [1994] propuseram a heurística da mediana e estabeleceram matematicamente sua garantia de aproximação de fator 3, consolidando o OSCM como um desafio de otimização combinatória de elevada restrição.

Em decorrência dessa complexidade computacional inerente e da multiplicidade de aplicações práticas, a literatura sobre desenho de grafos e minimização de cruzamentos ramificou-se

em diversas abordagens. A modelagem do problema, a topologia do espaço de busca e a viabilidade das heurísticas dependem fortemente de características estruturais e restrições específicas, as quais são comumente categorizadas na literatura segundo as seguintes dimensões:

Número de camadas e estrutura: Ambientes de visualização que lidam com grafos planares do tipo *st-graphs* [Tamassia, 1990], estruturas de duas camadas (bipartidas) [Eades and Wormald, 1994] ou desenhos de múltiplas camadas hierárquicas [Healy and Nikolov, 2013].

Densidade do grafo: O comportamento do espaço de busca varia se o grafo apresenta alta densidade de arestas [Eades and Wormald, 1994], ou se mantém características de esparsidade estrita, mantendo-se NP-difícil mesmo para florestas de estrelas Muñoz et al. [2001].

Objetivos: Minimizar puramente o número de cruzamentos de arestas [Garey and Johnson, 1983] ou uma combinação multi-objetivo envolvendo a redução de curvas em polilinhas (grafos com arestas assemelhando a polígonos) e a otimização da área ocupada pelo desenho no plano [Tamassia, 1990].

Restrições de posicionamento: Restrições de ordenamento parcial onde subconjuntos de vértices possuem precedência rígida de exibição [Sugiyama et al., 1981] e/ou vértices restritos a posições predefinidas em linhas paralelas fixas [Muñoz et al., 2001].

Padrões gráficos de desenho: Exibição através de arestas retilíneas no plano [Tutte, 1963] ou representações em malhas ortogonais e monotonicamente crescentes na direção vertical utilizando arcos especializados Tamassia [1990].

A viabilidade de soluções exatas para a minimização de cruzamentos esbarra em severos limites computacionais. Jünger and Mutzel [2002] mapearam essa restrição via programação inteira, demonstrando que grafos com ambas as camadas variáveis são tratáveis apenas em instâncias esparsas (até 15 vértices na menor camada). Focando na variante unilateral (*one-sided*), Kobayashi and Tamaki [2015] introduziram um algoritmo subexponencial de complexidade parametrizada (FPT) guiado pelo número de cruzamentos. Posteriormente, o compêndio de Schaefer [2018] consolidou essas bases estruturais, reafirmando a intratabilidade matemática inerente ao cálculo exato nesses grafos.

Dada a inviabilidade exata generalizada, algoritmos de aproximação tornaram-se a principal via analítica. Inicialmente, Demetrescu and Finocchi [2001] exploraram a equivalência entre a minimização de cruzamentos e a identificação de conjuntos mínimos de arcos de realimentação, desenvolvendo a estratégia de minimização de penalidades e evidenciando que a otimização isolada de cruzamentos pode degradar outras métricas do *layout*. Em seguida, visando superar o limite teórico clássico de fator 3 da heurística da mediana, Nagamochi [2005] propôs um algoritmo determinístico com garantia de aproximação superior, fixada em 1,4664.

No âmbito das heurísticas clássicas, Matuszewski et al. [2003] atestaram empiricamente a superioridade do método do baricentro iterado, consolidando-o como a principal linha de base para a qualidade de soluções. Simultaneamente, técnicas de refinamento local, notadamente o método de *sifting*, firmaram-se como mecanismos fundamentais e padrão para a otimização iterativa de vizinhanças.

Para instâncias de larga escala, a literatura contemporânea transita para metaheurísticas e modelos estocásticos. Recentes garantias teóricas foram propostas por Baumann et al. [2025], que provaram limites rigorosos de tempo de execução (como $\Theta(n^2)$) para operações de troca

adjacente. Em estudo complementar no mesmo ano, demonstraram que operadores de salto não adjacente aceleram substancialmente a convergência de algoritmos evolutivos no OSCM. Paralelamente, abordagens de aprendizado de máquina, como a aplicação de redes de grafos [Weidmann, 2020], despontam como novos paradigmas metodológicos, permitindo inferir ordenações de vértices de forma não puramente combinatória.

3.1 Análise Metodológica do PACE 2024

A relevância contemporânea e a dificuldade intrínseca do OSCM foram atestadas ao ser escolhido como o problema central da 9ª edição do *Parameterized Algorithms and Computational Experiments Challenge* [PACE, 2024]. Ao avaliar o estado da arte através de instâncias massivas, a competição evidenciou como a literatura atual lida com a fronteira da computabilidade em três trilhas de otimização: exata, parametrizada e heurística.

3.1.1 Trilha Exata

Na trilha focada na resolução exata, a principal constatação metodológica foi o abandono da busca direta no espaço de permutações em favor de reduções a problemas clássicos de otimização combinatória, resolvidas majoritariamente via programação linear inteira (PLI). A equipe MPPEG [Jünger et al., 2024], por exemplo, obteve destaque ao converter o OSCM no problema de ordenação linear (LOP), solucionando-o via técnicas de *branch-and-cut*. Paralelamente, a equipe UzL [Bannach et al., 2024] optou por uma redução ao problema do conjunto de arcos de realimentação direcionado (DFAS). De modo complementar, o *solver* CRGone [Dobler, 2024] consolidou-se unindo formulações PLI nativas a regras agressivas de pré-processamento e redução de instâncias. Conjuntamente, esses trabalhos comprovam que a viabilidade exata atual depende mais de formulações algébricas indiretas e *solvers* de alto desempenho do que da análise topológica pura.

3.1.2 Trilha Parametrizada

A trilha parametrizada — focada em instâncias com garantias estruturais, como baixa largura de corte — revelou uma dicotomia metodológica mais rica. Uma parcela dos competidores manteve a filosofia de redução: o algoritmo LUNCH [Langedal et al., 2024] mapeou o problema para satisfatibilidade booleana (SAT) baseando-se na divisão em subproblemas, enquanto a equipe MPPEG reaplicou sua arquitetura LOP. No entanto, esta trilha validou a eficácia da otimização topológica direta. O algoritmo OCMu64 [Groot Koerkamp and de Vries, 2024] destacou-se ao aplicar uma rotina de *branch-and-bound* estritamente sobre a ordem nativa dos vértices, viabilizada por novos teoremas de redução local. Essa divisão indica que, sob garantias paramétricas, a poda estrutural direta no grafo original torna-se tão competitiva quanto abstrações algébricas.

3.1.3 Trilha Heurística: Estratégias de Busca e Metaheurísticas

Na trilha heurística, o paradigma de avaliação altera-se drasticamente: impõe-se um limite rigoroso de tempo de execução de 5 minutos por instância e abdica-se da prova de otimalidade estrita em prol da máxima redução de cruzamentos viável dentro desse teto computacional. Este cenário é particularmente crítico para a fundamentação desta pesquisa, pois atesta que a limitação de tempo em instâncias de larga escala inviabiliza o uso de métodos exatos puros na

indústria, forçando a adoção de estratégias de busca local e metaheurísticas estocásticas. A análise minuciosa dos algoritmos que dominaram o topo do ranqueamento revela uma polarização em três frentes metodológicas, descritas a seguir.

1. Reduções Estruturais e Abstrações Lógicas

Uma parcela das equipes optou por adaptar arquiteturas concebidas para o cálculo exato limitando computacionalmente o seu espaço de busca. O algoritmo *LUNCH* [Langedal et al., 2024], que conquistou o 2º lugar na competição, por exemplo, manteve a estratégia de reduzir o OSCM à identificação de conjuntos de arcos de realimentação direcionado (DFAS). Para atuar em ambiente heurístico, o modelo foca em isolar eficientemente subproblemas independentes na topologia da rede, submetendo-os a *solvers* lógicos iterativos (MAXSAT) que são abortados prematuramente ao atingir limites de tolerância. Trata-se de uma abordagem matemática robusta, porém suscetível ao alto custo temporal de conversão das instâncias para o formato lógico.

2. Abordagens Híbridas de Busca Local

Outra vertente apostou na simplificação inicial do grafo combinada com buscas combinatórias táticas, consolidando as metodologias que ocuparam do terceiro ao quinto lugar. Destaca-se nesta categoria a abordagem de Geiger Geiger [2024], que garantiu a 3ª colocação integrando busca local iterada (ILS) com busca de vizinhança variável (VNS). Após aplicar regras de pré-processamento, o autor utilizou a heurística do baricentro para gerar uma solução inicial rápida. A otimização ocorreu refinando a solução iterativamente ao mover nós individuais ou blocos de até 5 vértices consecutivos. Para escapar de ótimos locais, o método introduziu uma perturbação agressiva, invertendo até 20% do conteúdo das soluções.

Compartilhando dessa filosofia base, o algoritmo *Arcee* [Boehmer et al., 2024] (4º lugar) e a implementação da equipe *Shadoks* [D. da Fonseca, 2024] (5º lugar) priorizaram a aplicação intensiva de regras de redução nativas (aglutinando ou fixando vértices triviais) diretamente na formulação original do OSCM. Após a redução do grafo, o núcleo residual é atacado por rotinas de busca local. O diferencial competitivo do *Arcee* residiu na sua integração de técnicas explícitas para escapar de mínimos locais através de perturbações pontuais no ordenamento posicional, em vez de depender de ferramentas comerciais de resolução.

3. Metaheurísticas Populacionais e Gestão de Diversidade

A verdadeira fronteira de escalabilidade nesta trilha foi dominada por arquiteturas estocásticas altamente sofisticadas, consagradas pelo grande vencedor da competição, o modelo *MAEDM-OCM* da equipe CIMAT [Segura et al., 2024]. Este sistema estrutura-se como um algoritmo memético que mescla componentes evolutivos com busca local iterada (ILS). O sucesso da ferramenta decorre da sua gestão explícita de diversidade: o modelo utiliza estratégias de sobrevivência *best-non-penalized* e operadores de *cycle crossover* que previnem ativamente o algoritmo de estagnar prematuramente em ótimos locais. Na etapa de intensificação, o modelo foca-se exclusivamente em operadores táticos de deslocamento (*shifts*), restringindo a avaliação apenas a um subconjunto de vizinhos. Ao processar instâncias gigantescas (acima de 17.000 vértices móveis), o *MAEDM-OCM* desativa por completo a sua base evolutiva-genética devido à explosão do custo computacional de memória, operando puramente via busca local iterada a partir de uma solução gulosa fortemente heurística.

Capítulo 4

Materiais e Métodos

Neste capítulo, são formalizados os conceitos do OSCM, detalhando suas propriedades matemáticas, complexidade e os desafios associados à sua resolução. Adicionalmente, apresenta-se a base conceitual do método de resolução adotado, o PT, descrevendo suas etapas de funcionamento.

4.1 Minimização de Cruzamentos Unilaterais

Seja $G = (V, E)$ um grafo bipartido simples, onde o conjunto disjunto de vértices é particionado em duas camadas, V_1 e V_2 , e $E \subseteq V_1 \times V_2$ representa o conjunto de arestas. No OSCM, assume-se que os vértices de V_1 e V_2 são posicionados em duas retas paralelas horizontais, tipicamente em $y = 1$ e $y = 0$, respectivamente. A posição horizontal de cada vértice é determinada por uma ordem linear σ , sendo a camada superior corresponde a sequência σ_1 e a camada inferior a sequência σ_2 .

Formalmente, de acordo com Eades and Wormald [1994], dadas duas arestas $e_1 = (u_1, v_1)$ e $e_2 = (u_2, v_2)$, com $u_1, u_2 \in V_1$ e $v_1, v_2 \in V_2$, um cruzamento ocorre se, e somente se, a ordem relativa de suas extremidades em V_1 é oposta à ordem relativa de suas extremidades em V_2 . Assumindo que $\sigma_1(u_1) < \sigma_1(u_2)$ (ou seja, u_1 está à esquerda de u_2), haverá um cruzamento se:

$$\sigma_2(v_2) < \sigma_2(v_1) \tag{4.1}$$

O problema tem como entrada um grafo bipartido $G = (V_1, V_2, E)$, onde a camada superior V_1 possui uma ordem linear fixa $\sigma_1 = (a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10})$, enquanto a ordem σ_2 da camada V_2 ainda não foi estabelecida. A Figura 4.1 ilustra uma instância típica de entrada baseada em PACE [2024], cuja disposição inicial arbitrária segue a sequência $\sigma_2 = (b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8, b_9, b_{10})$, resultando em 33 cruzamentos para apenas 12 arestas, gerando uma alta densidade de cruzamentos que prejudica a legibilidade do diagrama.

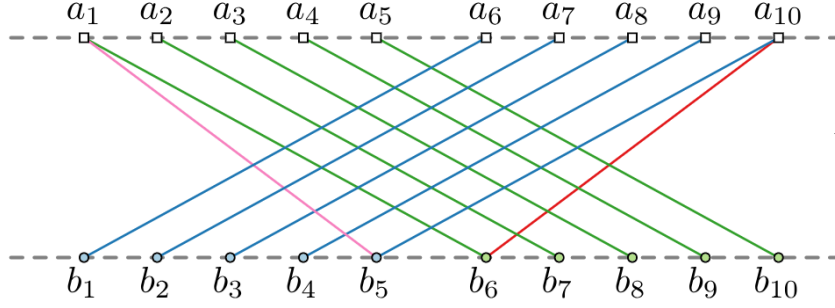


Figura 4.1: Exemplo de diagrama de entrada para o problema OSCM com ordenação arbitrária em V_2 . Adaptado de PACE [2024].

Uma solução para o OSCM é uma permutação dos vértices de V_2 , ou seja, σ_2 . O número total de cruzamentos C pode ser calculado pela soma dos cruzamentos entre todos os pares de arestas. Conforme estabelecido por Eades and Wormald [1994], para cada par de vértices $\{u, v\} \subseteq V_2$, define-se o número de cruzamentos c_{uv} como o número de cruzamentos que ocorrem se u estiver posicionado à esquerda de v . O valor de c_{uv} é dado por:

$$c_{uv} = \sum_{w \in N(u), z \in N(v)} \mathbb{I}(\sigma_1(z) < \sigma_1(w)) \quad (4.2)$$

onde $N(u)$ representa o conjunto de vizinhos do vértice u na camada fixa V_1 , e $\mathbb{I}$ é uma função indicadora que assume valor 1 se a condição for verdadeira. Na Figura 4.2, é apresentada a solução ótima obtida após a reordenação da camada inferior, cuja sequência passa a ser $\sigma_2 = (b_6, b_7, b_8, b_9, b_{10}, b_1, b_2, b_3, b_4, b_5)$, nota-se que essa nova permutação minimiza o total de cruzamentos para 17.

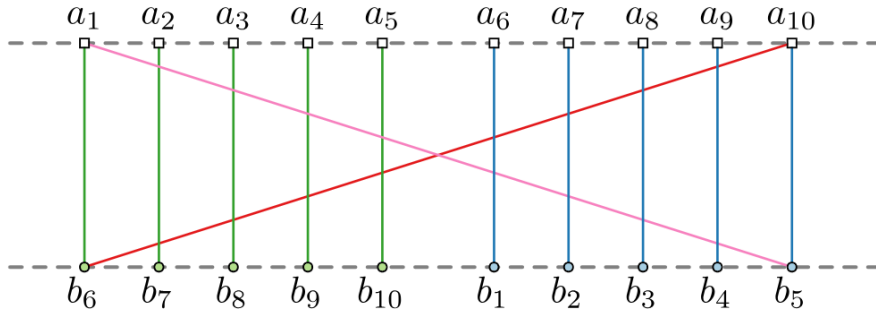


Figura 4.2: Diagrama resultante após a otimização da ordem de V_2 , apresentando o número mínimo de cruzamentos. Adaptado de PACE [2024].

O objetivo do OSCM é encontrar uma ordem linear σ_2 para os vértices da camada inferior V_2 que minimize c_{uv} . O desafio computacional reside no fato de que o OSCM é um problema de complexidade NP-difícil [Eades and Wormald, 1994]. Pertencer a essa classe de complexidade implica que não se conhece um algoritmo determinístico em tempo polinomial capaz de encontrar a solução ótima global para grafos arbitrários [Garey and Johnson, 1979]. Embora a minimização de cruzamentos possa parecer intuitiva, o espaço de busca pela solução cresce fatorialmente em relação ao número de vértices de V_2 .

4.2 Revenimento paralelo

O matemático Stanisław Ulam concebeu a ideia de que problemas matemáticos e determinísticos de alta complexidade poderiam ser aproximados por meio de simulações estatísticas baseadas em amostragens aleatórias repetidas. Com o refinamento e a formalização matemática conduzidos em parceria com John von Neumann, essa abordagem estocástica foi batizada como o método de Monte Carlo (MC). A resolução de problemas através de métodos de MC consiste basicamente em simulações independentes dentro de um espaço amostral de tamanho n , cuja convergência ao ótimo teórico é garantida quando n tende ao infinito [Benov, 2016].

Contudo, se a amostragem for realizada de maneira puramente independente e sem considerar a topologia do espaço amostral, essa convergência torna-se proibitivamente lenta em dimensões elevadas. Para acelerar esse processo e viabilizar a exploração eficiente de espaços amostrais massivos, o método MC evoluiu com a integração das cadeias de Markov, dando origem aos métodos MCMC [Jones and Qin, 2022]. Sob essa nova abordagem, ao invés de as amostras serem geradas de forma isolada, um mecanismo estocástico transforma a configuração atual s em uma nova configuração s' .

Essa transição possui a propriedade Markoviana de “memória curta”, dependendo exclusivamente do estado atual e da probabilidade de transição entre os estados. Essa dinâmica garante que, após um período de aquecimento, a cadeia convirja para uma distribuição estacionária que se alinha à distribuição alvo, concentrando a busca nas regiões mais promissoras do espaço de estados [Jones and Qin, 2022]. Para mapear com precisão os estados do sistema, os métodos MCMC frequentemente fundamentam-se na distribuição de Boltzmann, um conceito oriundo da mecânica estatística para descrever a probabilidade de o sistema se encontrar em uma determinada configuração.

É nesse ponto que se introduz o algoritmo de Metropolis, que avalia apenas a probabilidade de transição entre a configuração atual s e uma configuração candidata s' pertencente à sua vizinhança [Almeida, 2024]. Quando a transição proposta resulta em uma melhoria ou igualdade no valor de avaliação, a nova configuração é aceita de forma determinística. Por outro lado, caso resulte em uma piora nos critérios de avaliação, a nova configuração não é descartada sumariamente, ao invés disso, a aceitação torna-se probabilística e passa a seguir a função de distribuição de Boltzmann. Formalmente, o critério de aceitação do algoritmo de Metropolis é segmentado em dois casos e definido conforme a Equação (4.3).

$$P(\Delta E, T) = \begin{cases} 1, & \text{se } \Delta E \leq 0 \\ e^{-\frac{\Delta E}{T}}, & \text{se } \Delta E > 0 \end{cases} \quad (4.3)$$

Nessa equação, a variável ΔE representa a variação de energia observada durante a transição de estados, sendo calculada a partir da diferença entre os valores de avaliação $E()$ da nova configuração proposta e do estado atual, de modo que $\Delta E = E(s') - E(s)$. A constante T , chamada de temperatura, é um valor parametrizável que atua como um coeficiente de controle abstrato para regular a probabilidade de aceitação de novos estados com pior valor de avaliação ao longo do processo de busca. Sob altas temperaturas, o fator de aceitação aproxima-se de ($P \rightarrow 1$), estimulando a exploração e diversificação do espaço amostral, já à medida que a temperatura T diminui, o critério torna-se mais restritivo, convergindo para um comportamento puramente guloso focado na intensificação ao redor das melhores configurações encontradas.

Quando os métodos MCMC e o algoritmo de Metropolis são formalmente transpostos do domínio da mecânica estatística para o cenário da Otimização Combinatória, realiza-se um mapeamento direto entre os sistemas físico e computacional [Almeida, 2024]. Sob essa perspectiva, as configurações do espaço de estados passam a representar soluções viáveis do problema

combinatório; a energia E do sistema passa a indicar o valor de custo ou a avaliação ($f(s)$) de uma solução; e o parâmetro abstrato de temperatura T mantém o papel de coeficiente de controle entre diversificação e intensificação da busca por soluções.

Apesar de robusto, o algoritmo de Metropolis sofre frequentemente com o problema de estagnação em ótimos locais quando opera sob uma única temperatura fixa. Se o sistema for configurado sob um patamar térmico baixo, o critério torna-se estrito e a cadeia fatalmente ficará aprisionada por barreiras energéticas elevadas, por outro lado, se mantido em altas temperaturas, o comportamento puramente aleatório impede a convergência e o refinamento das soluções.

Como uma solução elegante a esse impasse, o método PT, formalizado originalmente sob o nome de *replica exchange*, propõe uma dinâmica fundamentada no uso de um conjunto de κ cadeias de Markov independentes em paralelo, onde cada réplica r_i é associada a uma temperatura constante T_i , $i = 1, \dots, \kappa$. Periodicamente, o algoritmo introduz um mecanismo de troca de réplicas entre temperaturas adjacentes i e $i + 1$. A aceitação dessa troca é regulada por um critério estocástico, cuja probabilidade é calculada pela Equação (4.4).

$$P(r_i \rightarrow r_{i+1}) = \min[1, \exp(\Delta\beta \cdot \Delta E)] \quad (4.4)$$

Na referida equação, $\Delta\beta = \frac{1}{T_j} - \frac{1}{T_i}$ representa a diferença entre os inversos das temperaturas associadas às réplicas e ΔE define a variação da função de avaliação entre as duas soluções. Esse arranjo confere ao algoritmo uma trajetória flexível e bidirecional no espaço de temperaturas, permitindo que uma réplica momentaneamente estagnada em uma temperatura baixa suba para um patamar térmico elevado, transponha barreiras enérgicas através da diversificação, e retorne posteriormente a níveis inferiores de temperatura para realizar uma intensificação refinada.

Historicamente, o PT foi proposto de forma quase simultânea no início da década de 1990 em duas frentes de conhecimento distintas: na simulação estocástica de sistemas complexos por Geyer [1991] e no estudo físico de vidros de *spin* (*spin-glass*) por Hukushima and Nemoto [1996]. Embora tenha se consolidado rapidamente como ferramenta padrão nas áreas de química quântica e biologia molecular, sua transposição para o campo da Otimização Combinatória e Pesquisa Operacional permaneceu restrita por décadas, consistindo em uma importante lacuna metodológica [Almeida, 2024].

A utilização do PT em problemas de otimização provou-se altamente vantajosa quando comparada a metaheurísticas clássicas baseadas em trajetória. Enquanto métodos tradicionais perdem a capacidade de escape de ótimos locais à medida que o tempo de execução avança, o PT mantém o equilíbrio dinâmico entre diversificação e intensificação durante toda a execução através do deslocamento de soluções no espectro de temperaturas diferentes [Almeida, 2024]. Essa característica confere ao PT a propriedade de ser um método estatisticamente exato, apresentando convergência superior e resultados robustos em superfícies de resposta altamente não convexas e repletas de ótimos locais.

A dinâmica genérica do PT é apresentada no Algoritmo 4.1. Inicialmente, define-se o conjunto de temperaturas $\mathcal{T}$ e atribui-se uma réplica r_i com uma solução inicial para cada temperatura T_i (linhas 1 e 2). Sob um laço interno (linhas 6 a 15), o algoritmo executa a fase de exploração local, na qual cada réplica r_i progride independentemente através de uma cadeia de Markov de comprimento L . A cada passo, um vizinho aleatório é avaliado pelo critério de aceitação (Equação 4.3) para atualizar o estado de r_i , atualizando-se também a melhor solução encontrada. Concluídos as cadeias de Markov, a fase de troca de réplicas (linhas 17 a 24) introduz coordenação global ao propor trocas estocásticas entre réplicas de temperaturas adjacentes, guiadas pela Equação 4.4. Esse processo se repete até que algum dos critérios de parada seja atingido, retornando a melhor solução encontrada em todas as réplicas.

Algoritmo 4.1: Framework PT

```
1 Inicializar conjunto de temperaturas  $\mathcal{T} = \{T_1, T_2, \dots, T_L\}$  conforme distribuição predefinida;  
2 Atribuir uma réplica por temperatura com uma solução inicial;  
3 Definir a quantidade máxima de trocas de réplica a serem propostas;  
4 enquanto critérios de parada não atendidos faça  
5     // Fase de Exploração Local (Cadeias de Markov)  
6     para cada  $T_i \in \mathcal{T}$  faça  
7         para passo  $j \leftarrow 1$  até comprimento da cadeia de Markov  $L$  faça  
8             Gerar solução vizinha aleatória na réplica atual  $r_i$ ;  
9             Aplicar critério de aceitação (Equação 4.3);  
10            se vizinho for aceito então  
11                Atualizar a réplica  $r_i$  com a nova solução;  
12            fim  
13        fim  
14        Atualizar a melhor solução encontrada;  
15    fim  
16    // Fase de tentativa de trocas de réplicas  
17    para cada pares de temperaturas adjacentes  $(T_i, T_{i+1})$  faça  
18        Propor uma troca entre  $r_i$  e  $r_{i+1}$ ;  
19        Aplicar critério de aceitação (Equação 4.4);  
20        se a troca for aceita então  
21            Efetivar a troca de réplicas entre temperaturas;  
22        fim  
23    fim  
24 fim  
25 retorna a melhor solução encontrada em todas as réplicas;
```

A estrutura em réplicas independentes revela que o PT possui uma natureza intrinsecamente paralela. Para viabilizar a aplicação desse algoritmo em problemas de PO sem a complexidade de gerenciar a concorrência diretamente, uma interface de programação de aplicações genérica (API) *multi-core* foi desenvolvida por Almeida et al. [2025] e disponibilizada abertamente em <https://github.com/ALBA-CODES/PTAPI>.

A PT API isola completamente as rotinas dependentes do problema, como a função de avaliação, os operadores de vizinhança, as estruturas de representação das soluções e os mecanismos de validação de restrições. Desta forma, reduz-se drasticamente o esforço de codificação de novas abordagens heurísticas de alto desempenho baseadas no PT. Contudo, essa integração está longe de ser um processo puramente automático ou trivial; o acoplamento de uma nova aplicação exige uma modelagem estrutural criteriosa. É necessário mapear o domínio discreto do problema para as interfaces da API, implementar estratégias eficientes de movimentação no espaço de busca e, frequentemente, codificar heurísticas de construção para a geração de soluções iniciais viáveis.

Assim, embora a API forneça a implementação de todo o motor genérico, o gerenciamento das réplicas e a infraestrutura de controle térmico, o acoplamento e o desempenho da metaheurística dependem diretamente desse trabalho especializado de customização para o problema abordado. O próximo capítulo descreve os detalhes do desenvolvimento deste acoplamento do PT ao OSCM.

Capítulo 5

Desenvolvimento

Este capítulo detalha a modelagem, as decisões algorítmicas e a implementação computacional desenvolvidas para solucionar o OSCM. A solução fundamenta-se na adoção, adaptação e extensão da API [Almeida et al., 2025] voltada para a execução do PT.

5.1 A API de Revenimento Paralelo

Para sustentar o desenvolvimento de uma busca paralela e eficiente no espaço de soluções do OSCM, utilizou-se a infraestrutura da API concebida para o PT. Esta API provê uma implementação multi-*core* em CPU focada na linguagem C++, desenhada com o propósito de oferecer uma interface flexível para uso em diferentes plataformas e integrar os recursos de computação de alto desempenho à pesquisa operacional.

A escolha analítica por esta arquitetura baseia-se na sua alta capacidade de separação de responsabilidades. A API abstrai o gerenciamento do paralelismo, controlando de forma nativa a orquestração das réplicas simultâneas e os parâmetros térmicos. O núcleo do *framework* gerencia o tamanho da cadeia de Markov homogênea simulada em cada réplica e os esquemas de distribuição de temperatura inicial. O algoritmo subjacente também se encarrega de aplicar o critério de Metrópolis para a aceitação estocástica de soluções e das trocas de informações entre temperaturas adjacentes. Além disso, a API oferece métodos autônomos para a atualização do conjunto de temperaturas durante a execução do método.

Com a metodologia da busca estocástica totalmente encapsulada, o esforço de desenvolvimento pôde ser focado exclusivamente nas particularidades do problema, o que a API define como as etapas principais de acoplamento.

5.2 Codificação e Decodificação

A etapa de codificação atua para mapear a entrada do problema para uma representação computacional que o método consiga operar de forma eficiente. Uma solução candidata é codificada por um vetor unidimensional de inteiros, o qual dita uma permutação específica dos vértices da camada livre do grafo bipartido. Cada elemento dessa estrutura armazena o identificador real do nó, guiado pela formulação $nA + 1 + i$ para garantir a clara separação lógica em relação aos vértices fixos. Essa representação também encapsula o custo avaliado da permutação, permitindo que o algoritmo transite entre os estados de busca sem incorrer em reprocessamentos constantes dos cruzamentos.

O processo de decodificação é acionado sempre que a API necessita extrair o resultado dessa abstração numérica. Essa tradução ocorre em memória para projetar a ordenação atual sobre a estrutura do grafo e calcular com exatidão os cruzamentos geométricos das arestas.

5.3 Heurística Construtiva da Solução Inicial

A etapa de construção atua como o ponto de partida para a exploração do espaço de soluções, encarregada de instanciar uma solução base para cada réplica que será operada pelo algoritmo PT. A implementação foi arquitetada para ser flexível, permitindo que a estratégia de inicialização seja definida dinamicamente com base nas configurações individuais de cada réplica.

Dependendo da estratégia selecionada, o método popula a solução injetando a ordenação previamente calculada pela heurística do Baricentro [Sugiyama et al., 1981] ou pela heurística da Mediana [Eades and Wormald, 1994]. A heurística do Baricentro atua calculando, para cada vértice da camada livre, a média aritmética das posições de todos os seus vizinhos conectados na camada fixa. De forma parecida, a heurística da Mediana avalia essas mesmas conexões, mas extrai o valor da mediana das posições adjacentes. Em ambas as abordagens, a solução primária é estruturada organizando os vértices livres em ordem crescente com base nos respectivos índices geométricos calculados.

Por fim, caso a parametrização do sistema estipule uma inicialização estocástica, ou na simples ausência de uma configuração específica para a réplica em questão, o algoritmo contorna o uso dessas heurísticas e gera uma permutação puramente aleatória dos vértices livres para iniciar o processo de otimização.

5.4 Operadores de Perturbação e Vizinhança

O algoritmo seleciona estocasticamente dois índices distintos na permutação de nós livres e aplica um dos três operadores de vizinhança implementados, descritos brevemente a seguir:

Swap: Troca direta de posições entre os dois índices selecionados.

Insert: Remoção de um vértice de sua posição original para inserção em uma nova.

2-opt: Inversão completa da ordem de um subconjunto contíguo da permutação, localizado entre os dois índices selecionados.

A implementação suporta a flexibilidade de fixar o uso de um único operador ou de intercalar entre os três de forma aleatória a cada nova tentativa de aplicação.

5.5 Funções de Avaliação

A contagem de cruzamentos entre as arestas é a métrica central do OSCM e, consequentemente, a operação mais custosa de todo o algoritmo. Em grafos bipartidos densos, recalculá-lo a cada pequena alteração na ordem dos vértices tornaria a execução de métodos estocásticos inviável do ponto de vista do tempo de processamento. Para contornar esse gargalo e sustentar a alta taxa de iterações exigida pelo PT, o mecanismo de avaliação foi dividido em duas abordagens distintas e complementares: uma avaliação global e exata para validações sistêmicas, e uma avaliação parcial e rápida para a transição contínua entre os estados de busca.

5.5.1 Funções de Avaliação Completa

A avaliação completa atua como o validador definitivo do problema, encarregado de retornar a quantidade estrita de cruzamentos de uma determinada solução inteira. A primeira camada de otimização desse método é um simples mecanismo de cache: se o custo da solução já foi computado incrementalmente pela avaliação parcial durante a fase de geração de vizinhança, o método retorna imediatamente esse valor, contornando qualquer processamento adicional.

Para os cenários em que a avaliação global é inescapável (como na construção das soluções iniciais), o método foi construído com garantias rigorosas de concorrência. Utilizando uma instância avaliadora local para cada *thread*, assegura-se que as réplicas trabalhando em paralelo não corrompam o estado do grafo umas das outras ao alterarem a ordem dos vértices. Uma vez que a permutação é carregada de forma segura para o avaliador interno da *thread*, a contagem total é delegada a uma implementação estruturada em *Fenwick Tree* (ou *Segment Tree*), que resolve o cálculo global de cruzamentos de forma matematicamente exata e veloz.

5.5.2 Avaliação Parcial

A viabilidade de executar milhares de movimentos a cada execução da metaheurística é atribuída ao procedimento de avaliação parcial. Em vez de acionar uma recontagem completa iterativamente, o método extrai apenas a diferença local de custo (δ) gerada pela troca, utilizando consultas diretas à matriz de cruzamentos. Ao subtrair e somar estritamente os custos relativos à área afetada pela movimentação dos vértices, o algoritmo atualiza o valor objetivo da nova solução, reduzindo drasticamente a carga de processamento exigida.

Essa matriz de cruzamentos, denotada por $V_{f_{n \times n}}$ (em que n representa a quantidade de vértices livres), constitui o cerne da otimização de desempenho, viabilizando consultas da avaliação parcial em tempo constante, $O(1)$. Em vez de instanciar varreduras sobre o grafo, a matriz armazena a diferença exata no número de cruzamentos resultante da troca de posição entre dois vértices livres específicos.

Devido ao consumo massivo de memória RAM exigido por instâncias de grafos de grande porte, a instanciação dessa estrutura foi projetada de forma dinâmica. Durante a inicialização, o algoritmo analisa os recursos do hardware, estima um limite seguro de memória disponível e calcula a dimensão máxima suportada para a matriz. Caso o tamanho do grafo exceda a capacidade física da máquina, o sistema aloca apenas o quadrante suportado e realiza o cálculo de forma puramente analítica caso os pares solicitados pelos movimentos caiam fora dessa área de cobertura.

Adicionalmente, a estrutura opera sob o paradigma de avaliação preguiçosa. Inicialmente, todas as coordenadas recebem um marcador de ausência de dados. O processamento analítico do custo para um par de vértices (u, v) só é executado no exato momento em que o algoritmo requisita essa informação pela primeira vez. Após o cálculo, o resultado é armazenado na respectiva coordenada e, tirando proveito da simetria do movimento de troca, o custo reverso e negativo é automaticamente inserido na coordenada espelhada (v, u) , reduzindo o esforço global de preenchimento da matriz pela metade.

5.6 Distribuição Inicial de Temperaturas

O sucesso do PT depende da configuração térmica, delimitada entre uma temperatura mínima ($T_{\min}$) e uma máxima ($T_{\max}$), que dita a probabilidade de aceitação de soluções pelo critério de

Metrópolis. Para estruturar o estado inicial do sistema, a API permite alocar os patamares térmicos através de esquemas distintos: a distribuição linear, que estabelece incrementos constantes entre as réplicas adjacentes; a linear-inversa, que aplica passos uniformes sobre o inverso da temperatura; a exponencial, que modula os intervalos com base em uma função de decaimento contínuo; e a geométrica, na qual as temperaturas progridem por um fator multiplicativo estrito. Através da parametrização, o algoritmo garante que a distribuição térmica inicial seja delineada para se adequar à complexidade de cada instância antes do início da execução.

5.7 Ajuste Dinâmico de Temperaturas

Embora a distribuição inicial defina a arquitetura térmica base, a evolução da busca frequentemente exige adaptações para manter a eficiência do PT. Para suprir essa necessidade, a API incorpora mecanismos de ajuste dinâmico de temperaturas em tempo de execução. Esse processo atua como um sistema de controle de malha fechada, monitorando continuamente o comportamento das réplicas, principalmente a taxa de sucesso nas tentativas de trocas de réplicas entre temperaturas adjacentes.

Com base nesse monitoramento contínuo, a arquitetura da API disponibiliza três abordagens principais para a recalibragem da malha: a otimização por *feedback* [Katzgraber et al., 2006], a equalização das taxas de aceitação de trocas entre temperaturas adjacentes [Hukushima, 1999] e a fixação da mesma taxa na meta teórica de 23% [Gelman et al., 1997, Roberts and Rosenthal, 2001].

A estratégia de otimização por *feedback* utiliza o histórico de aceitações em tempo real para identificar e corrigir gargalos estocásticos, ajustando o distanciamento térmico dinamicamente para melhorar a transição de soluções onde o fluxo apresenta quedas de rendimento. A abordagem de equalização, por sua vez, reconfigura iterativamente os patamares térmicos com o objetivo de nivelar a probabilidade de aceitação de trocas entre todos os pares de temperaturas adjacentes, garantindo uma difusão randômica e homogênea das soluções por todo o espectro. Por fim, a configuração de 23% ancora o mecanismo a um modelo matemático estrito, calibrando as temperaturas até que as taxas de intercâmbio converjam para esse percentual, o qual é amplamente validado na literatura de métodos de Monte Carlo como o limiar ótimo para maximizar a eficiência estatística da exploração do espaço de estados [Atchadé et al., 2011, Tawn et al., 2020].

5.8 Critério de Parada

A definição de critérios de parada é fundamental no projeto de metaheurísticas para garantir que o algoritmo conclua a exploração e retorne a melhor solução encontrada dentro de limites práticos de processamento computacional. Na implementação desenvolvida, o encerramento do PT é orquestrado de forma dinâmica por uma combinação de três condições distintas e complementares, sendo a busca finalizada assim que qualquer uma delas for satisfeita.

O primeiro critério atua como uma clássica barreira fundamentada em um limite de tempo de execução. Essa restrição cronológica é de vital importância para adequar a solução a cenários de avaliação competitiva (como as restrições de tempo impostas pelo desafio PACE [2024]), garantindo que o algoritmo interrompa a exploração em todas as *threads* e decodifique a melhor solução global assim que o tempo limite estipulado for alcançado.

O segundo parâmetro de controle baseia-se no limite de ciclos completos de busca que o sistema está autorizado a realizar, servindo como a condição matemática primária para a com-

pletude do algoritmo. A busca é finalizada naturalmente caso a contagem global de iterações alcance esse teto predefinido.

Por fim, o terceiro critério avalia a estagnação da busca com base em uma métrica de parada antecipada por percentual de convergência. Durante a execução, o algoritmo monitora ativamente em qual iteração específica a melhor solução global foi atualizada pela última vez. O algoritmo calcula uma margem de tolerância baseada em uma fração do limite de iterações do método. Se a exploração avançar por essa quantidade de passos estipulada sem registrar nenhuma redução no número de cruzamentos, o algoritmo detecta a convergência para um ótimo local e aborta o processamento para evitar o desperdício de recursos computacionais.

Capítulo 6

Resultados e Discussão

Neste capítulo, são apresentados os experimentos computacionais conduzidos para validar a abordagem proposta frente ao problema de minimização de cruzamentos em grafos bipartidos de camada livre. A avaliação contempla a consistência, escalabilidade e capacidade de convergência do algoritmo PT comparativamente ao algoritmo genético (GA) desenvolvido por Segura et al. [2024], método vencedor do desafio PACE.

Os testes computacionais foram executados em uma máquina equipada com processador Intel Core i5-10400 CPU @ 2.90GHz (8 núcleos lógicos), 16 GB de memória RAM, sob o sistema operacional Ubuntu 21.10. O código foi implementado em C++ e compilado via g++ 11.2.0 com a flag de otimização -O3.

6.1 Instâncias

A Tabela 6.1 sintetiza a diversidade dimensional das 600 instâncias consideradas, as quais foram disponibilizadas pelo PACE [2024] e estão divididas igualmente em 3 conjuntos de 200 instâncias. São detalhados a quantidade de vértices na camada fixa (V_0), na camada livre (V_1) e o total de arestas (E).

Tabela 6.1: Resumo das características estruturais dos conjuntos de instâncias avaliados.

Categoria	Qtd. Instâncias	Vértices Fixos (V_0)	Vértices Livres (V_1)	Total de Arestas (E)
		[Min – Máx]	[Min – Máx]	[Min – Máx]
Exata	200	276 – 16.543	261 – 16.148	112 – 32.807
Parametrizada	200	772 – 7.464	780 – 7.320	1.135 – 31.692
Heurística	200	256 – 131.315	256 – 130.809	763 – 1.114.112

6.2 Ajuste de Parâmetros e Protocolo Experimental

A calibração dos parâmetros do PT foi executada via pacote *irace* López-Ibáñez et al. [2016], dividindo o conjunto de treinamento pelas dimensões dos grafos em instâncias pequenas (P) e grandes (G). A Tabela 6.2 sumariza os domínios explorados e a configuração de elite adotada.

Tabela 6.2: Domínios de busca do *irace* e configuração de elite validada e adotada nos testes.

Parâmetro	Espaço de Busca	Configuração adotada
Temperatura inicial ($T_{\min}$)	{0.01; 1.0; 3.0; 5.0}	0.01
Temperatura final ($T_{\max}$)	{10; 30; 50; ...; 1000}	30 (P) / 300 (G)
Comprimento da cadeia (MCL)	{200; 300; 500}	500
Laços do <i>Parallel Tempering</i> (PTL)	{50; 100; 150; 200}	100
Distribuição de temperatura	{Linear, Linear-inversa, Exponencial, Geométrica}	Geométrica
Critério de atualização	{Desabilitado, 23%, Taxas Iguais, <i>Feedback</i> }	23%
Intervalo de atualização	{10; 20; ...; 120}	30 (P) / 50 (G)
Estrutura de vizinhança	{ <i>Swap</i> , <i>Insert</i> , 2-Opt, Aleatório}	<i>Insert</i>
Soluções iniciais (<i>setRep</i>)	Quantidade por heurística construtiva	1 Baricentro, 2 Medianas

A paralelização foi configurada para utilizar 4 réplicas ativas (metade das *threads* lógicas). O tempo limite nominal de execução foi fixado em 5 minutos (300 segundos) para replicar o ambiente de competição do PACE [2024]. Adicionalmente, definiu-se um cenário de execução estendida por 1 hora (3600 segundos) para mapear o comportamento de convergência em longo prazo.

6.3 Avaliação das Instâncias Exatas

A Tabela 6.3 consolida os indicadores computacionais para as 200 instâncias exatas. Na referida tabela, a coluna “método” indica o método e o regime de execução adotado, o número de cruzamentos médio obtidos, o desvio padrão para o índice anterior, o *gap* ou distância percentual em relação ao GA, o número médio de iterações (PTL) e o tempo de execução, em segundos. O *gap* relativo (%) entre a solução obtida pelo PT (*target*) e a solução de referência do GA (*reference*) é dada pela Equação 6.1:

$$gap = \left(\frac{target - reference}{reference} \right) \cdot 100,0 \quad (6.1)$$

Tabela 6.3: Desempenho comparativo entre o PT (5 min e 1 hora) e o algoritmo genético no conjunto de instâncias exatas.

Método	Cruzamentos (Média)	Desvio Padrão	<i>Gap</i> Médio (%)	PTL Médio	Tempo (s)
PT (Proposto - 5 min)	145.798,98	170.501,81	0,03	88.524,71	300,07 ± 0,11
PT (Proposto - 1 hora)	145.700,16	170.335,70	0,002	1.062.823,06	3600,06 ± 0,10
GA (Execução local - 5 min)	145.690,55	170.314,73	–	–	295,01 ± 0,01

A Figura 6.1 ilustra a comparação das melhores soluções obtidas por cada método em cada regime de execução, para todo o conjunto de instâncias, em ordem crescente pelo número de arestas.

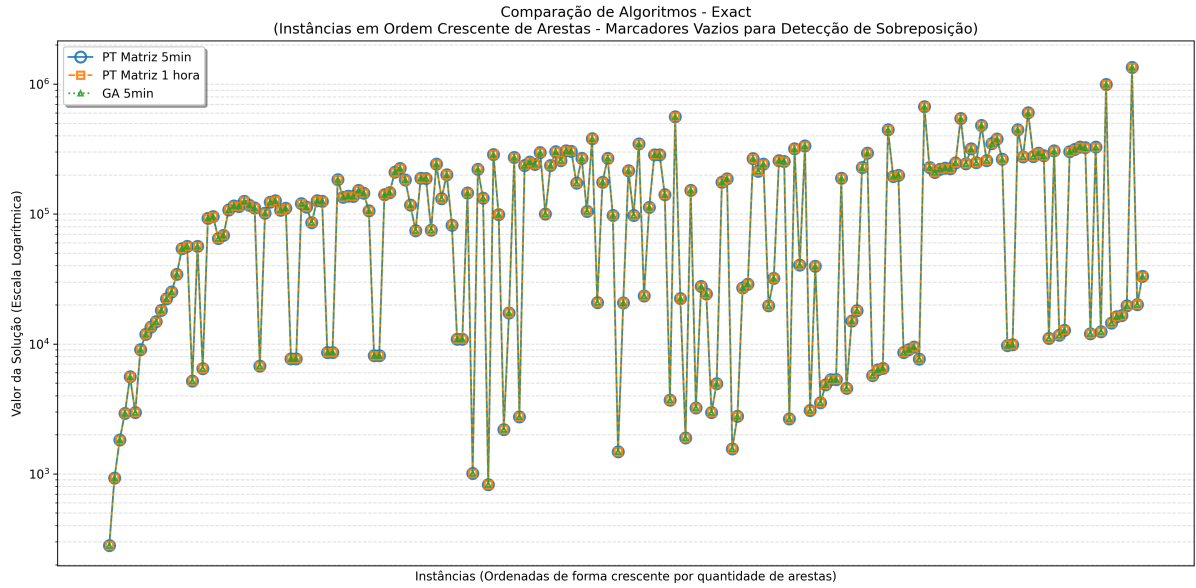


Figura 6.1: Comparação de soluções no conjunto exato.

Na regime nominal de 5 minutos, o PT empata rigorosamente com o GA em 141 das 200 instâncias (70,5%), o supera em 1 instância (0,5%) e apresenta desempenho inferior em 58 (29,0%), registrando um *gap* médio residual de 0,03%. Quando o tempo de execução do PT é estendido para 1 hora, o número de empates sobe para 146 (73,0%), mantendo 1 solução superior e reduzindo as perdas para 53 (26,5%). Esse esforço adicional comprime o *gap* médio para 0,002% e equaliza os desvios padrões entre os métodos, demonstrando que a troca térmica em cadeias mais longas supera barreiras locais em grafos de porte médio.

Observou-se ainda forte correlação negativa entre a quantidade de arestas do grafo e o número de iterações realizadas ($r = -0,78$). Grafos maiores aumentaram o custo computacional por iteração da estrutura de vizinhança *Insert*, reduzindo o volume de trocas por segundo, mas mantendo a estabilidade temporal do algoritmo.

6.4 Avaliação das Instâncias Parametrizadas

Seguindo o mesmo padrão de apresentação anterior a Tabela 6.4 e a Figura 6.2 detalham os resultados no agrupamento parametrizado.

Tabela 6.4: Desempenho comparativo entre o PT (5 min e 1 hora) e o algoritmo genético no conjunto de instâncias parametrizadas.

Método	Cruzamentos (Média)	Desvio Padrão	Gap Médio (%)	PTL Médio	Tempo (s)
PT (Proposto - 5 min)	81.598,92	122.734,96	0,004	45.767,93	300,06 ± 0,06
PT (Proposto - 1 hora)	81.587,96	122.693,25	0,0002	514.186,22	3600,07 ± 0,06
GA (Execução local - 5 min)	81.587,87	122.693,05	—	—	295,01 ± 0,01

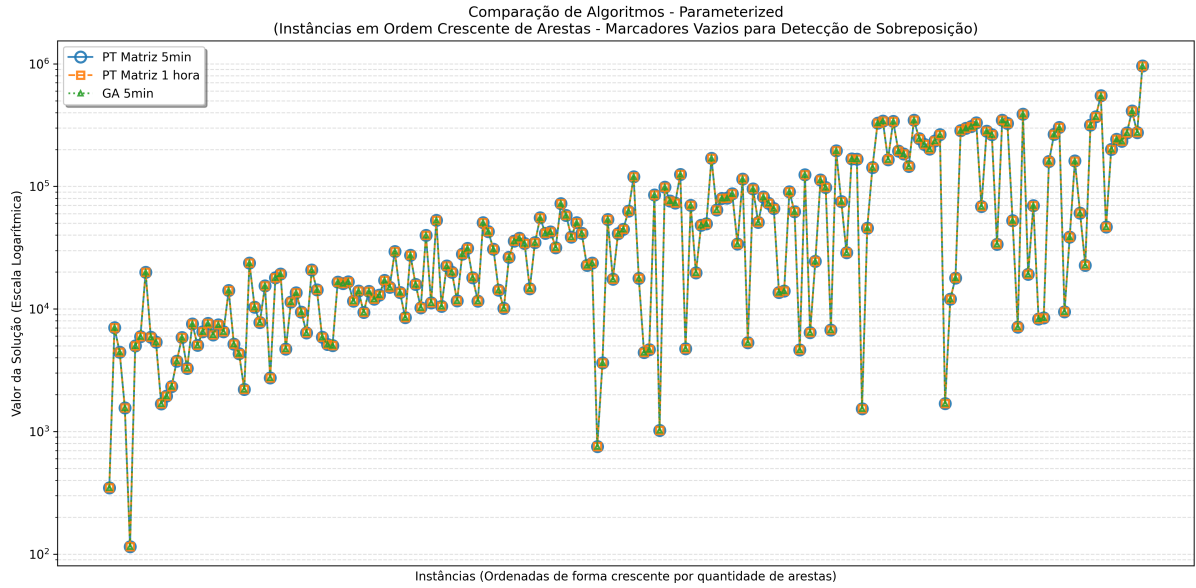


Figura 6.2: Comparação de soluções no conjunto Parametrizado, ordenadas por quantidade de arestas.

O conjunto parametrizado evidenciou o maior grau de equivalência e convergência entre as duas metaheurísticas comparadas. No regime de execução de 5 minutos, o algoritmo alcança a exata mesma solução do GA em 184 das 200 instâncias (92,0%), divergindo em apenas 16 (8,0%) com um *gap* médio de 0,004%. Com a expansão do tempo para 1 hora, o PT iguala a solução de referência do GA em 197 instâncias (98,5%), restando divergências residuais em apenas 3 grafos (1,5%) e reduzindo o *gap* médio global a 0,0002%.

A correlação entre a quantidade de arestas e o total de *PTL* alcançado foi de $r = -0,68$, corroborando que a menor densidade média desse conjunto permitiu maior frequência de trocas entre cadeias do que no conjunto Exato, garantindo rápida estagnação em soluções de alta qualidade.

6.5 Avaliação das Instâncias Heurísticas

Novamente, adota-se o mesmo padrão de apresentação de resultados. A Tabela 6.5 e a Figura 6.3 sintetizam o desempenho na classe de maior escala e densidade.

Tabela 6.5: Desempenho comparativo entre o PT e o algoritmo genético no conjunto de instâncias heurísticas.

Método	Cruzamentos (Média)	Desvio Padrão	Gap Médio (%)	PTL Médio	Tempo (s)
PT (Proposto - 5 min)	1.594.813.533,15	18.868.482.095,25	2,22	51.738,72	300,47 ± 2,45
PT (Proposto - 1 hora)	1.594.541.523,59	18.866.233.169,51	0,11	594.897,02	3600,39 ± 1,87
GA (Execução local - 5 min)	1.593.589.604,55	18.853.869.493,56	—	—	295,02 ± 0,04

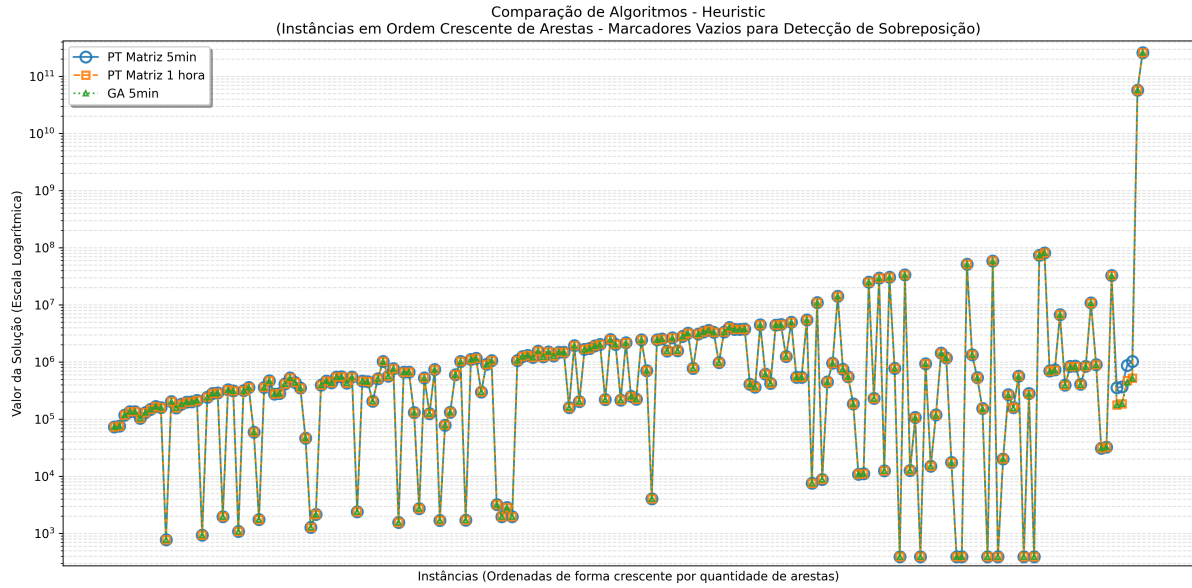


Figura 6.3: Comparação de soluções no conjunto Heurístico, ordenadas por quantidade de arestas.

O comportamento no conjunto heurístico expõe a sensibilidade da busca perante dimensões massivas (alcançando mais de 1,1 milhão de arestas). Em 5 minutos, a restrição temporal limita a varredura do PT, resultando em 75 empates (37,5%) e 125 soluções piores (62,5%), elevando o *gap* médio para 2,22%. No entanto, o cenário de 1 hora ilustra a capacidade de recuperação do método: os empates sobem para 89 (44,5%) e o *gap* médio se torna 0,11%. Os elevados desvios padrão observados em ambos os métodos decorrem da grande dispersão na escala dos grafos da categoria (variando de centenas a bilhões de cruzamentos totais). A correlação entre *PTL* e número de arestas ajustou-se para $r = -0,20$, reflexo de instâncias extremas onde o gargalo de memória e varredura de adjacências domina a complexidade de cada ciclo.

6.6 Discussão Integrada e Análise de Escalabilidade

A Tabela 6.6 consolida a distribuição comparativa das 600 instâncias analisadas, permitindo mapear o comportamento global do PT frente ao GA.

Tabela 6.6: Distribuição sintética dos confrontos diretos entre PT e GA nas 600 instâncias do benchmark.

Conjunto de Dados	Tempo limite	Empates (%)	PT Melhor (%)	PT Pior (%)	Gap Médio (%)
Exato (200)	5 min	141 (70,5%)	1 (0,5%)	58 (29,0%)	0,030%
	1 hora	146 (73,0%)	1 (0,5%)	53 (26,5%)	0,002%
Parametrizado (200)	5 min	184 (92,0%)	0 (0,0%)	16 (8,0%)	0,004%
	1 hora	197 (98,5%)	0 (0,0%)	3 (1,5%)	0,0002%
Heurístico (200)	5 min	75 (37,5%)	0 (0,0%)	125 (62,5%)	2,220%
	1 hora	89 (44,5%)	0 (0,0%)	111 (55,5%)	0,105%
Total Geral (600)	5 min	400 (66,7%)	1 (0,2%)	199 (33,1%)	0,751%
Total Geral (600)	1 hora	432 (72,0%)	1 (0,2%)	167 (27,8%)	0,036%

A síntese dos experimentos demonstra que o GA apresenta vantagem em cenários com restrição severa de tempo devido aos seus operadores de recombinação populacional, que realizam saltos globais no espaço de busca. Por outro lado, o PT provou ser uma alternativa altamente robusta e de implementação paralelizável eficiente: no horizonte nominal de 5 minutos, igualou ou superou o GA em 66,8% de todas as instâncias avaliadas, com *gap* médio global inferior a 0,76%. Ao expandir a janela para 1 hora, essa taxa de equivalência/superioridade elevou-se para 72,2%, reduzindo o *gap* médio global para 0,036%.

Capítulo 7

Conclusões

Neste estudo, foram abordadas a formalização, o desenvolvimento e a avaliação experimental da metaheurística *parallel tempering* (PT) aplicada ao problema de minimização de cruzamentos em grafos bipartidos de camada livre. Esse problema possui elevada relevância prática e teórica em áreas como visualização de redes e biologia computacional, nas quais a clareza visual e a redução de intersecções impactam diretamente a interpretabilidade das estruturas. A literatura recente demanda algoritmos paralelos robustos, capazes de explorar eficientemente espaços de busca complexos e escaláveis para instâncias massivas, como as propostas no desafio oficial do *PACE Challenge 2024*.

Os experimentos computacionais conduzidos considerando 600 instâncias demonstraram que a abordagem proposta oferece resultados promissores e altamente competitivos perante o algoritmo genético (GA) vencedor da competição. No regime restrito a 5 minutos de execução, o PT igualou ou superou a solução do GA em 66,8% de todo o conjunto de testes, mantendo um *gap* médio global de apenas 0,751%. Ao estender o tempo para 1 hora, a dinâmica de trocas térmicas entre as réplicas evitou a estagnação prematura em ótimos locais, elevando a taxa de equivalência para 72,2% das instâncias e reduzindo o *gap* médio para 0,036%, com destaque para a classe parametrizada, que atingiu 98,5% de soluções equivalentes ao estado da arte.

Trabalhos futuros se concentrarão no desenvolvimento de um critério de parada adaptativo baseado no monitoramento da taxa de aceitação de trocas e da estagnação da energia entre as réplicas, visando interromper as execuções assim que a convergência for detectada, evitando o desperdício de recurso computacional. Pretende-se também explorar o uso de estruturas de vizinhança híbridas ou compostas, alternando dinamicamente os operadores de movimento conforme a densidade e o tamanho do grafo.

Além disso, a estrutura do algoritmo poderá ser estendida para a minimização de cruzamentos em grafos multicamadas (*k-Layer Crossing Minimization*), permitindo aplicar a metodologia em problemas de *layout* ainda mais complexos. Por fim, pretende-se submeter os achados e as contribuições deste trabalho em formato de artigo científico para apresentação no Simpósio Brasileiro de Pesquisa Operacional (SBPO).

Referências Bibliográficas

- André Luís Barroso de Almeida. *Revisitando o revenimento paralelo: computação de alto desempenho e aplicação em pesquisa operacional*. Tese (doutorado em ciência da computação), Universidade Federal de Ouro Preto, Ouro Preto, 2024. URL <https://www.repositorio.ufop.br/handle/123456789/18678>.
- André Luís Barroso Almeida, Joubert de Castro Lima, and Marco Antonio Moreira Carvalho. Revisiting the parallel tempering algorithm: High-performance computing and applications in operations research. *Computers & Operations Research*, 178:107000, 2025. ISSN 0305-0548. doi: <https://doi.org/10.1016/j.cor.2025.107000>. URL <https://www.sciencedirect.com/science/article/pii/S0305054825000280>.
- Yves F Atchadé, Gareth O Roberts, and Jeffrey S Rosenthal. Towards optimal scaling of metropolis-coupled markov chain monte carlo. *Statistics and Computing*, 21(4):555–568, 2011.
- Max Bannach, Florian Chudigiewitsch, Kim-Manuel Klein, and Marcel Wienöbst. Pace solver description: Uzl exact solver for one-sided crossing minimization. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 28–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Jakob Baumann, Ignaz Rutter, and Dirk Sudholt. Analysing the effectiveness of mutation operators for one-sided bipartite crossing minimisation. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 872–880, 2025.
- Dobriyan M. Benov. The manhattan project, the first electronic computer and the monte carlo method. *Monte Carlo Methods and Applications*, 22(1):73–79, 2016. doi: [doi:10.1515/mcma-2016-0102](https://doi.org/10.1515/mcma-2016-0102). URL <https://doi.org/10.1515/mcma-2016-0102>.
- Kimon Boehmer, Lukas Lee George, Fanny Hauser, and Jesse Palarus. Pace solver description: Arcee. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 33–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Guilherme D. da Fonseca. Shadoks pace 2024 solver, June 2024. URL <https://doi.org/10.5281/zenodo.11452654>.
- Camil Demestrescu and Irene Finocchi. Breaking cycles for minimizing crossings. *Journal of Experimental Algorithmics (JEA)*, 6:2–es, 2001.
- Alexander Dobler. Pace solver description: Crgone. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 29–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

- Peter Eades and Nicholas C Wormald. Edge crossings in drawings of bipartite graphs. *Algorithmica*, 11(4):379–403, 1994.
- David J Earl and Michael W Deem. Parallel tempering: Theory, applications, and new perspectives. *Physical Chemistry Chemical Physics*, 7(23):3910–3916, 2005.
- Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, pages 128–140, 1741.
- M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*. W. H. Freeman, first edition edition, 1979. ISBN 0716710455. URL <http://www.amazon.com/Computers-Intractability-NP-Completeness-Mathematical-Sciences/dp/0716710455>.
- Michael R Garey and David S Johnson. Crossing number is np-complete. *SIAM Journal on Algebraic Discrete Methods*, 4(3):312–316, 1983.
- Martin Josef Geiger. Pace solver description: Martin_j_geiger. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 32–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Andrew Gelman, Walter R Gilks, and Gareth O Roberts. Weak convergence and optimal scaling of random walk metropolis algorithms. *The annals of applied probability*, 7(1):110–120, 1997.
- Charles J Geyer. Markov chain monte carlo maximum likelihood. 1991.
- Ragnar Groot Koerkamp and Mees de Vries. Pace solver description: Ocmu64, a solver for one-sided crossing minimization. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 35–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Ulrich HE Hansmann. Parallel tempering algorithm for conformational studies of biological molecules. *Chemical Physics Letters*, 281(1-3):140–150, 1997.
- Frank Harary. Determinants, permanents and bipartite graphs. *Mathematics magazine*, 42(3):146–148, 1969.
- Frank Harary. A new crossing number for bipartite graphs. *Utilitas Math*, 1:203–209, 1972.
- Patrick Healy and Nikola S. Nikolov. Hierarchical drawing algorithms. In Roberto Tamassia, editor, *Handbook of Graph Drawing and Visualization*, chapter 13, pages 409–453. CRC Press, 2013. URL <https://cs.brown.edu/people/rtamassi/gdhandbook/chapters/hierarchical.pdf>.
- Koji Hukushima. Domain-wall free energy of spin-glass models: Numerical method and boundary conditions. *Physical Review E*, 60(4):3606, 1999.
- Koji Hukushima and Koji Nemoto. Exchange monte carlo method and application to spin glass simulations. *Journal of the Physical Society of Japan*, 65(6):1604–1608, 1996.

- Galin L. Jones and Qian Qin. Markov chain monte carlo in practice. *Annual Review of Statistics and Its Application*, 9(Volume 9, 2022):557–578, 2022. ISSN 2326-831X. doi: <https://doi.org/10.1146/annurev-statistics-040220-090158>. URL <https://www.annualreviews.org/content/journals/10.1146/annurev-statistics-040220-090158>.
- Michael Jünger and Petra Mutzel. 2-layer straightline crossing minimization: Performance of exact and heuristic algorithms. In *Graph algorithms and applications i*, pages 3–27. World Scientific, 2002.
- Michael Jünger, Paul J Jünger, Petra Mutzel, and Gerhard Reinelt. Pace solver description: Exact solution of the one-sided crossing minimization problem by the mpeg team. *arXiv preprint arXiv:2412.00292*, 2024.
- Helmut G Katzgraber, Simon Trebst, David A Huse, and Matthias Troyer. Feedback-optimized parallel tempering monte carlo. *Journal of Statistical Mechanics: Theory and Experiment*, 2006(03):P03018–P03018, 2006.
- Yasuaki Kobayashi and Hisao Tamaki. A fast and simple subexponential fixed parameter algorithm for one-sided crossing minimization. *Algorithmica*, 72(3):778–790, 2015.
- Kenneth Langedal, Matthias Bentert, Thorgal Blanco, and Pål Grønås Drange. Pace solver description: Lunch-linear uncrossing heuristics. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 34–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016. ISSN 2214-7160. doi: <https://doi.org/10.1016/j.orp.2016.09.002>. URL <https://www.sciencedirect.com/science/article/pii/S2214716015300270>.
- Christian Matuszewski, Robby Schönfeld, and Paul Molitor. Using sifting for k-layer straightline crossing. In *Graph Drawing: 7th International Symposium, GD’99, Stirin Castle, Czech Republic, September 15-19, 1999 Proceedings*, page 217. Springer, 2003.
- Perna Mishra, Santosh Kumar, and Mithilesh Kumar Chaube. Graph interpretation, summarization and visualization techniques: A review and open research issues. *Multimedia Tools and Applications*, 82(6):8729–8771, 2023.
- Xavier Muñoz, Walter Unger, and Imrich Vrt’o. One sided crossing minimization is np-hard for sparse graphs. In *International Symposium on Graph Drawing*, pages 115–123. Springer, 2001.
- Hiroshi Nagamochi. An improved bound on the one-sided minimum crossing number in two-layered drawings. *Discrete & Computational Geometry*, 33(4):569–591, 2005.
- PACE. Parameterized algorithms and computational experiments. <https://pacechallenge.org/>, 2024.
- Regis Pires Magalhaes, Luís Gustavo Coutinho do Rêgo, José Macêdo, and Vânia Vidal. *Processamento de Grafos em Big Data*, pages 8–38. 10 2015. ISBN 978-85-99961-18-6.

- Gareth O Roberts and Jeffrey S Rosenthal. Optimal scaling for various metropolis-hastings algorithms. *Statistical science*, 16(4):351–367, 2001.
- Marcus Schaefer. *Crossing numbers of graphs*. CRC Press, 2018.
- Carlos Segura, Lázaro Lugo, Gara Miranda, and Edison David Serrano Cárdenas. Pace solver description: Cimat_team. In *19th International Symposium on Parameterized and Exact Computation (IPEC 2024)*, pages 31–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2): 109–125, 1981. doi: 10.1109/TSMC.1981.4308636.
- Roberto Tamassia. Drawing algorithms for planar st-graphs. *Australasian Journal of Combinatorics*, 2:217–235, 1990.
- Nicholas G Tawn, Gareth O Roberts, and Jeffrey S Rosenthal. Weight-preserving simulated tempering. *Statistics and Computing*, 30(1):27–41, 2020.
- William Thomas Tutte. How to draw a graph. *Proceedings of the London Mathematical Society*, 3(1):743–767, 1963.
- John N Warfield. Crossing theory and hierarchy mapping. *IEEE Transactions on Systems, Man, and Cybernetics*, 7(7):505–523, 2007.
- Mark E Watkins. A special crossing number for bipartite graphs: a research problem. *Annals of the New York Academy of Sciences*, 175(1):405–410, 1970.
- Thomas Weidmann. Approximating one-sided crossing minimization with graph networks. 2020.